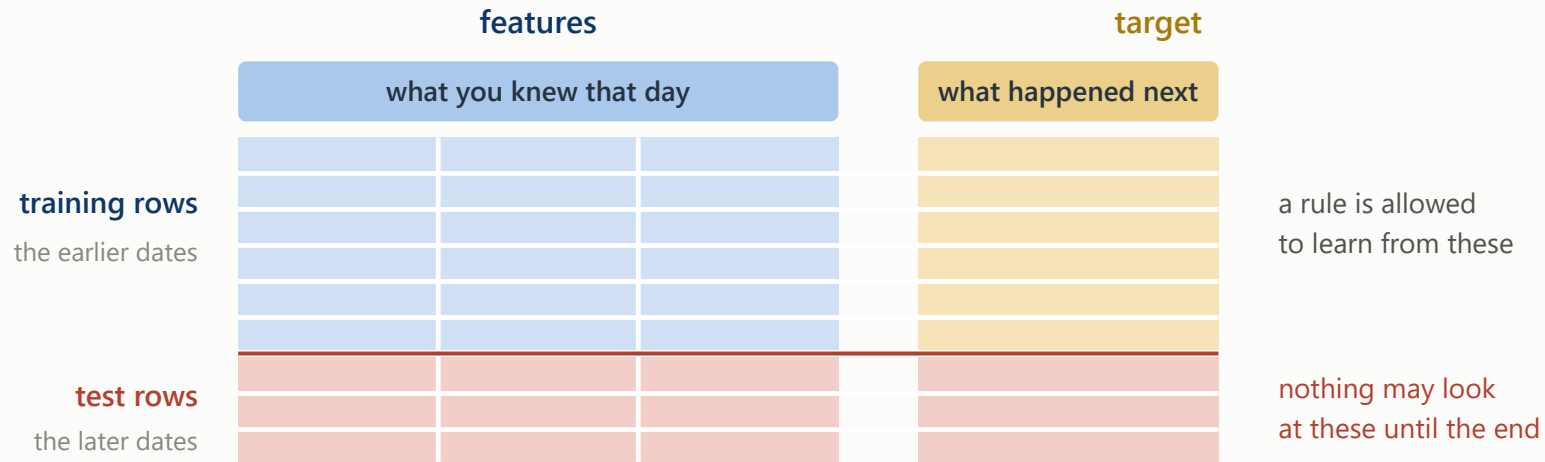


Model selection and cross-validation

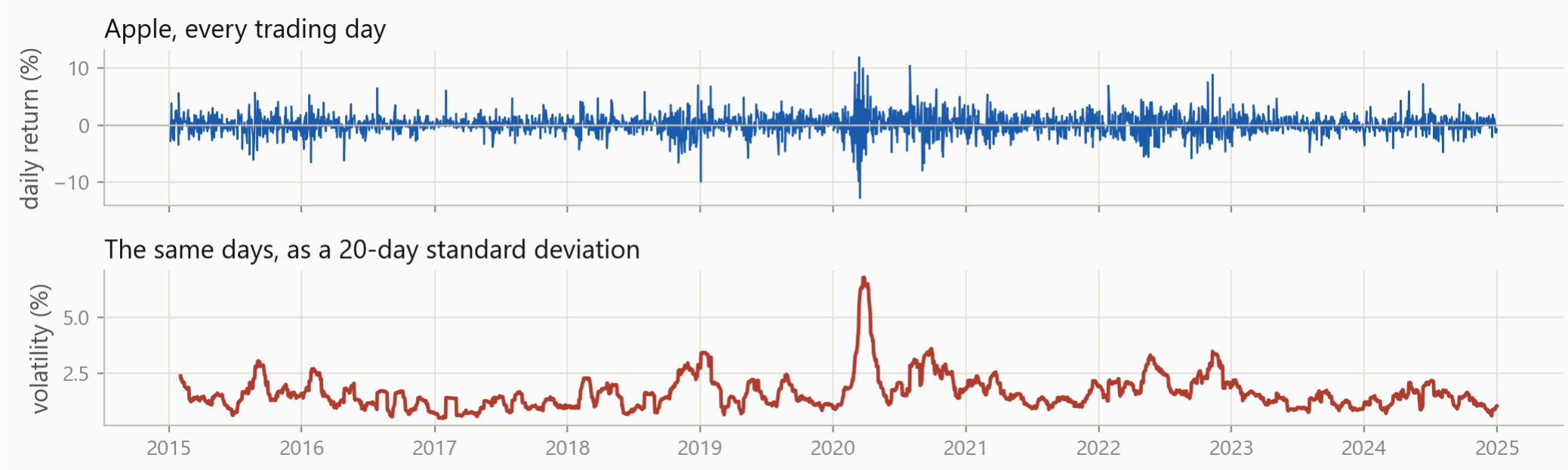
Machine Learning in Finance · Session 5

You have everything except a model



Last time you built a table like this and split it by date. This session fits a model to the training rows.

Some things are easier to predict than others



Returns are close to unpredictable. Volatility is not. **It clusters**: calm months are followed by calm months, so today's volatility says something about next month's.

Today

- fit a model with **scikit-learn** and score it on unseen days
- compare it with two rules that need no model at all
- see why the lowest error is often the wrong model to keep
- hold out a **validation** block, and see why one is not enough
- use **cross-validation**, with folds that respect the calendar

The lecture introduces each of these. The exercises and the case are where you practise them.

1 • Fitting a model

PART 1

Setting up the prediction problem, and fitting a linear regression to it.

The question

How volatile will Apple be over the next twenty trading days?

Why it matters

option prices, margin requirements and risk limits are all set from a volatility forecast

What you know today

how volatile the last twenty days were, how volatile the last sixty were, and how the stock has drifted

The answer is not known for twenty days, which makes this a forecast rather than a description.

The table

Python Code

```
prices = pd.read_csv("data/prices.csv", parse_dates=["date"])
rets = prices.pivot(index="date", columns="ticker", values="close").pct_change().dropna() * 100

r = rets["AAPL"]

table = pd.DataFrame({
    "vol_20d": r.rolling(20).std(),
    "vol_60d": r.rolling(60).std(),
    "ret_20d": r.rolling(20).mean(),
})
table["vol_next"] = r.rolling(20).std().shift(-20)
table = table.dropna()

table.head(3)
```

	vol_20d	vol_60d	ret_20d	vol_next
date				
2015-03-31	1.440084	1.753613	-0.184232	1.095689
2015-04-01	1.436198	1.708066	-0.159784	1.271219
2015-04-02	1.408131	1.709300	-0.033871	1.428156

rolling and **shift** were both covered last time. Multiplying by 100 converts the returns to percent, which keeps the numbers easier to read.

The three features and the target

Python Code

```
table.shape
```

```
(2436, 4)
```

looking back from today

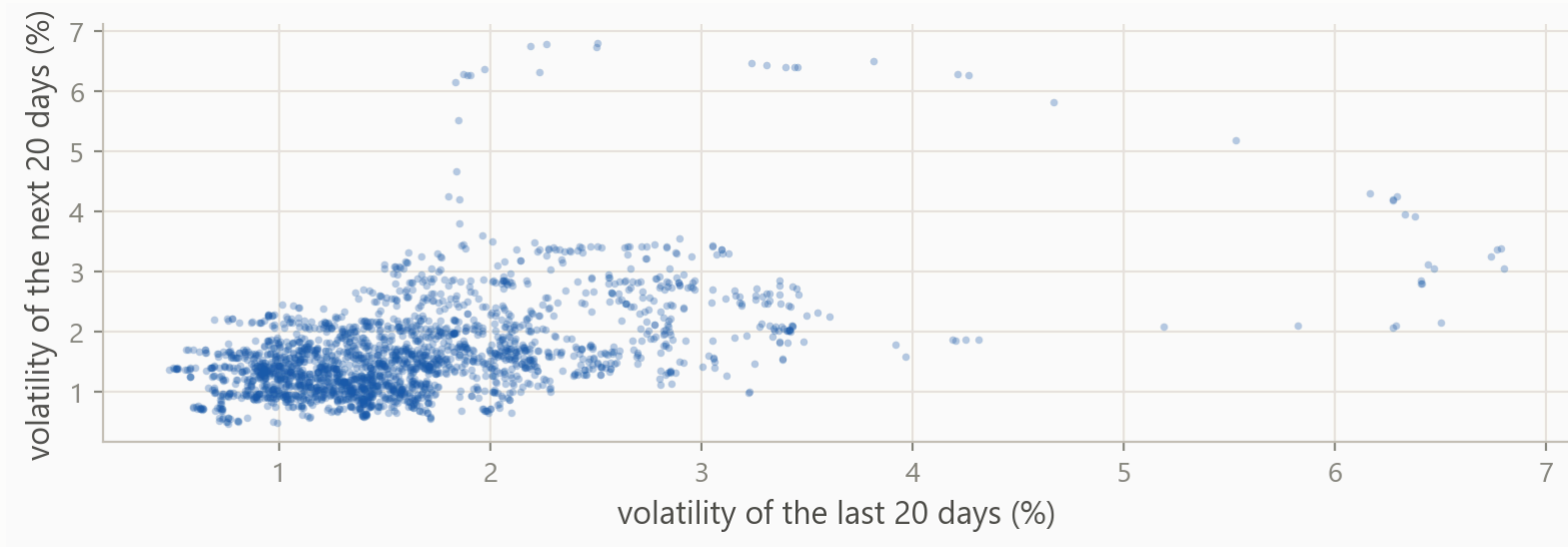
vol_20d vol_60d ret_20d

looking forward from today

vol_next

2,436 rows

Is there anything there?



Each dot is one day. The cloud slopes upwards, so a volatile month tends to be followed by another one. A linear regression finds the line through it.

Splitting by date

Python Code

```
train = table.loc[:"2022-12-31"]  
test = table.loc["2023-01-01":]  
  
print(len(train))  
print(len(test))
```

1954
482

Split on a **date**, not at random. A random test day would sit between two training days with almost identical volatility, so the score would come out too good.

X and y

scikit-learn takes the features and the target as two separate objects.

Python Code

```
X_train = train[["vol_20d"]]  
y_train = train["vol_next"]
```

```
X_train.head(3)
```

	vol_20d
date	
2015-03-31	1.440084
2015-04-01	1.436198
2015-04-02	1.408131

Two pairs of brackets: the inner pair is a **list of columns**, as in `prices[["date", "close"]]`. One pair returns a single column, which cannot be used as `X`.

scikit-learn

scikit-learn contains all sorts of models, and it is already installed in Colab.

Models

linear regression today; ridge, lasso and trees later in the course

Metrics

mean squared error, R^2 , and the other scoring measures covered last time

Splitting

tools for dividing the data into training, validation and test blocks

The package is installed under the name `scikit-learn` but imported as `sklearn`.

Import, create, fit

Python Code

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

print(model)
```

```
LinearRegression()
```

LinearRegression() creates a model that has not been fitted to anything yet. Calling **.fit()** with the training features and the training target computes the coefficients.

Printing the model shows only which kind of model it is. The coefficients it computed are stored on the object itself.

What it learned

Python Code

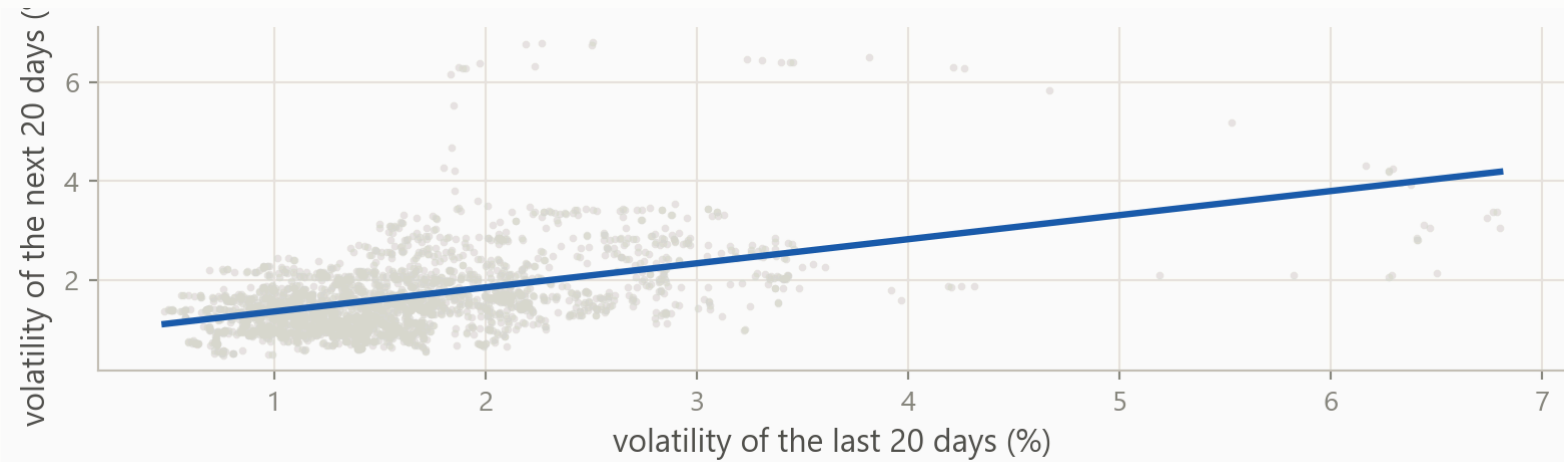
```
print(model.intercept_)  
print(model.coef_)
```

```
0.8812185871624809  
[0.48653208]
```

A trailing underscore marks an attribute **computed from the data**. `model.coef_` does not exist until `.fit()` has run.

Reading the two numbers

$$\widehat{\text{vol}}_{\text{next}} = 0.88 + 0.49 \times \text{vol}_{20d}$$



The slope is 0.49, not 1: only half of this month's excess volatility carries into next month. **Volatility is pulled back towards its average**, which is called mean reversion.

Predicting

Python Code

```
X_test = test[["vol_20d"]]  
  
predictions = model.predict(X_test)  
predictions[:5]
```

```
array([1.84353895, 1.86425406, 1.84318103, 1.94708334, 1.93549308])
```

`.predict()` returns one number per row of `X_test`. These are days in 2023 and 2024, none of which were used to fit the model.

The forecast, against what happened



The forecast tracks the broad moves but lags at every turn. Comparing it with anything else needs a number, not a picture.

2 · Scoring the model

PART 2

Measuring the error, and choosing something sensible to compare it with.

Mean squared error

Python Code

```
from sklearn.metrics import mean_squared_error  
  
mean_squared_error(test["vol_next"], predictions)
```

```
0.16451986221081497
```

Every metric in scikit-learn takes the true values first and the predictions second.

Root mean squared error

Python Code

```
rmse = np.sqrt(mean_squared_error(test["vol_next"], predictions))  
print(rmse)
```

```
0.40561048089369556
```

The square root puts the error back into percentage points: **the forecast is wrong by about 0.41 percentage points on a typical day.**

Is 0.41 good?

An RMSE of 0.41 does not mean anything on its own. To judge it, compare it against two rules that are so simple they need no model.

Guess the average

predict the same number every day and ignore the features entirely

Repeat what just happened

predict that the next twenty days will be as volatile as the last twenty

Neither rule has parameters and neither needs fitting. Together they set the minimum that a model has to beat.

Rule one: always guess the average

Python Code

```
guess = train["vol_next"].mean()
print(guess)

flat = [guess] * len(test)
print(np.sqrt(mean_squared_error(test["vol_next"], flat)))
```

1.7097010837987425

0.5196501142755454

The average is computed on the **training** rows. Using the test average would require information that was not available when the forecast was made.

Rule two: repeat this month

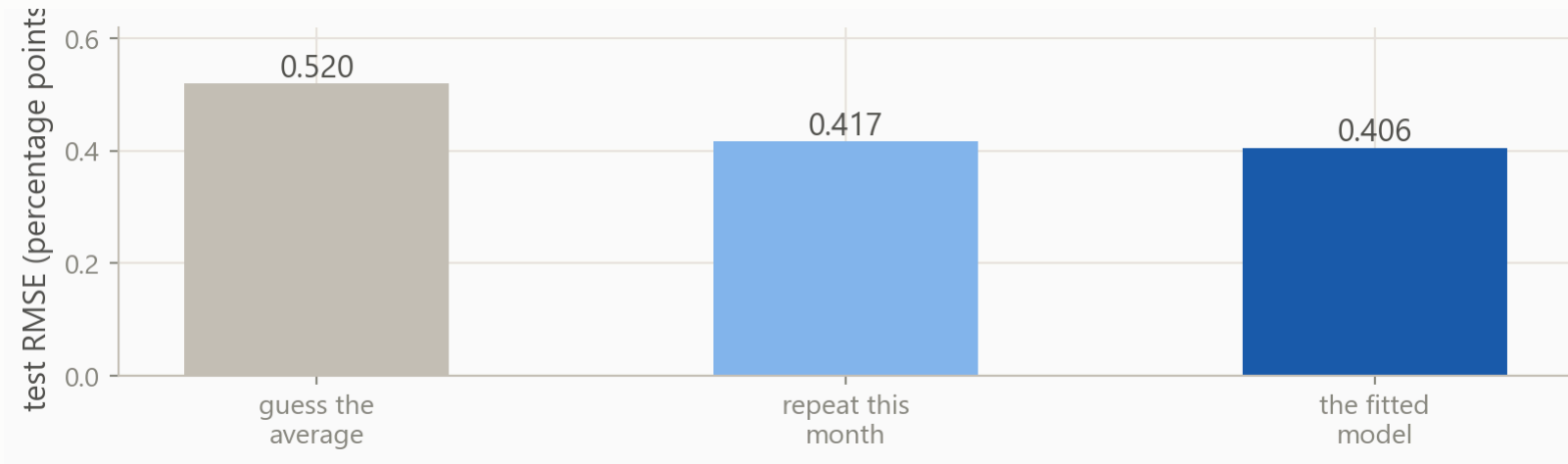
Python Code

```
print(np.sqrt(mean_squared_error(test["vol_next"], test["vol_20d"])))
```

```
0.4172768816166766
```

The forecast for the next twenty days is simply the last twenty, with no fitting involved. Because volatility clusters, it is already accurate.

The three forecasts compared



The model wins, but only by about three percent over a rule with no parameters.

Margins that small are normal in finance.

Your turn 1

Fit the same model on `vol_60d` instead of `vol_20d`, and score it the same way. Does the longer window predict better?

Python Code

```
long_model = LinearRegression()
long_model.fit(...)

long_pred = long_model.predict(test[["vol_60d"]])
print(np.sqrt(mean_squared_error(test["vol_next"], long_pred)))
```

`.fit()` takes two arguments, and the last two lines show what shape each one has.

Your turn 1: answer

```
long_model.fit(train[["vol_60d"]], train["vol_next"])
```

0.419, against 0.406 for **vol_20d**. A sixty-day average reacts more slowly, so it lags further at the turns.

The features come from **train**, not **test**, and need the same double brackets as **predict**. The target is one column, so it takes a single pair.

A natural next question is whether using both features together does better than either one alone.

3 • Choosing between models

PART 3

Comparing several models, and deciding which one to keep.

Three candidates

Python Code

```
A = ["vol_20d"]  
B = ["vol_20d", "vol_60d"]  
C = ["vol_20d", "vol_60d", "ret_20d"]
```

All three use the same model, the same rows and the same target. They differ only in which columns go into **X**.

Naming the three lists here lets the later slides write `train[A]` instead of repeating the column names.

Scoring on the training rows

Python Code

```
for columns in [A, B, C]:  
    m = LinearRegression()  
    m.fit(train[columns], train["vol_next"])  
    error = np.sqrt(mean_squared_error(train["vol_next"], m.predict(train[columns])))  
    print(len(columns), "features:", round(error, 4))
```

```
1 features: 0.7218  
2 features: 0.7213  
3 features: 0.7102
```

The error falls each time a column is added.

More columns always fit better

Adding a column can never worsen the fit. A useless column can be given a coefficient of zero, reproducing the previous fit exactly.

Training error therefore always favours the largest model. It measures the fit on these rows, not whether the extra columns help on new ones.

This is overfitting: a model with enough freedom fits the noise in its own training rows.

Scoring on the test rows

Python Code

```
for columns in [A, B, C]:  
    m = LinearRegression()  
    m.fit(train[columns], train["vol_next"])  
    error = np.sqrt(mean_squared_error(test["vol_next"], m.predict(test[columns])))  
    print(len(columns), "features:", round(error, 4))
```

```
1 features: 0.4056  
2 features: 0.4108  
3 features: 0.4255
```

The ranking reverses. **The one-feature model has the lowest test error**, so the two extra columns were fitting noise rather than signal.

So use the test rows to choose?

It is tempting. The test rows are the honest out-of-sample score, and they just gave a clear answer.

The problem is what the word **lowest** is doing. Three test errors were computed and the smallest was kept. Part of being smallest is being genuinely better, and part of it is luck.

The winner's 0.406 is therefore too good. It is the best of three draws, not an unbiased estimate of what the model will do next year.

It gets worse with every choice

Three candidates is mild. A real project compares far more, and every comparison is another look at the test rows.

Features

which columns to include, which to transform, how long a window to use

Settings

every model from here has them: ridge has a shrinkage parameter, a tree has a depth

And they multiply

ten settings tried on four feature sets is forty looks at the test rows

Nothing from the test rows ever entered the code. It entered through **your decisions**, which is the hardest kind of leakage to notice.

Choosing and reporting are different jobs

Choosing

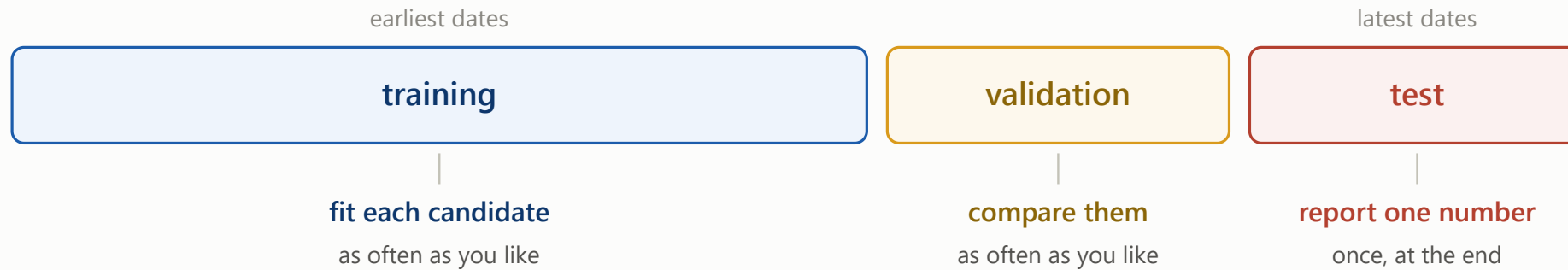
Comparing candidates and keeping one. This happens repeatedly, and every comparison uses up a little of the honesty of whatever rows it was done on.

Reporting

Measuring how well the chosen model does on rows that played no part in the choosing. This happens once, and that number is the one reported.

Two jobs means two blocks of data. Choosing needs held-out rows of its own.

A third block, for choosing



The training block is split again by date. The test block stays unused until the end.

Cutting the training block in two

Python Code

```
fit = train.loc[:"2020-12-31"]  
val = train.loc["2021-01-01":]  
  
print(len(fit))  
print(len(val))
```

```
1451  
503
```

Fit on everything to the end of 2020, then compare on 2021 and 2022. The test years remain unused.

Choosing on the validation block

Python Code

```
for columns in [A, B, C]:  
    m = LinearRegression()  
    m.fit(fit[columns], fit["vol_next"])  
    error = np.sqrt(mean_squared_error(val["vol_next"], m.predict(val[columns])))  
    print(len(columns), "features:", round(error, 4))
```

```
1 features: 0.5138  
2 features: 0.5155  
3 features: 0.5039
```

The **three-feature model wins**, and the test rows played no part in it. On the test rows the ranking was the other way round.

Your turn 2

Move the cut one year earlier. Fit on everything up to the end of **2019** and compare the candidates on 2020, 2021 and 2022.

Python Code

```
fit_early = train.loc[:...]
val_early = train.loc[...:]

for columns in [A, B, C]:
    m = LinearRegression()
    m.fit(fit_early[columns], fit_early["vol_next"])
    error = np.sqrt(mean_squared_error(val_early["vol_next"], m.predict(val_early[columns])))
    print(len(columns), "features:", round(error, 4))
```

Your turn 2: answer

```
fit_early = train.loc[:"2019-12-31"]  
val_early = train.loc["2020-01-01":]
```

The errors are **0.9978, 1.0025 and 1.0704**. The one-feature model now has the lowest error, by a clear margin.

The three candidates and the training block are unchanged. Only the cut date moved, and a different model came out best.

4 · One split is one experiment

PART 4

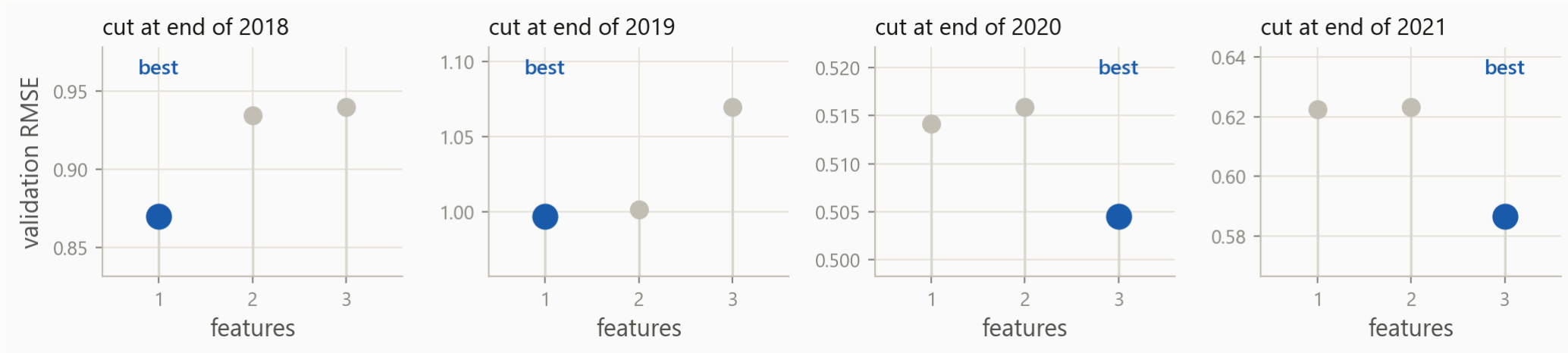
What happens when the validation block is placed somewhere else.

Two cuts, two winners

2021 and 2022 were chosen for convenience, not for any real reason. Moving the cut changed which model came out best.

When the cut date determines which model is kept, the result says more about the choice of cut date than about the models.

Every cut date, side by side



With the cut after 2018 or 2019 the one-feature model has the lowest error. With the cut after 2020 or 2021 the three-feature model does.

Each panel has its own scale, so compare the dots within a panel, not across them.

Why the winner changes

Each block is a single period

2020 contains a crash and 2021 does not, so the two periods favour different models

Each block is small

a few hundred rows, and consecutive rows overlap heavily because their windows do

Validation rows are not fitted on

every row held back for validation is a row the model does not learn from

The candidates differ by less than the periods do, so one cut date cannot separate them.

Using several cut dates

Since the cut dates disagree, there is no reason to trust any one of them. **Use several cut dates and average the results.**

Every cut is an honest measurement on its own. Averaging them stops any single period deciding the winner.

5 • Cross-validation

PART 5

Averaging over several cut dates, and reading the result.

Every row takes a turn

Split the training rows into K blocks, called **folds**. Each fold is held back once, scored by a model fitted without it, and the K scores are averaged.

K measurements, not one

the average of five is far steadier than any one of them, as the four cut dates already showed

And still fitted on

each row is used for fitting in the other folds, so none is permanently lost to validation

Every row is scored once

no row is left out of the judging, so no part of the sample is ignored

The spread comes free

K numbers say how much the answer moves from period to period, which one number cannot

The arithmetic

Write $\mathcal{F}_k$ for the rows scored in fold k , and n_k for how many there are:

$$e_k = \sqrt{\frac{1}{n_k} \sum_{i \in \mathcal{F}_k} (y_i - \hat{y}_i)^2} \qquad \text{CV}_K = \frac{1}{K} \sum_{k=1}^K e_k$$

- every $\hat{y}_i$ comes from a model fitted **without** fold k
- e_k is the RMSE on one fold, and CV_K is the average of the K of them
- drop the square root and CV_K is the textbook version, in MSE

Five cuts instead of one

	2015-16	2016-17	2017-19	2019-20	2020-21	2021-22	RMSE
cut 1	fit	score					0.583
cut 2	fit	fit	score				0.703
cut 3	fit	fit	fit	score			1.255
cut 4	fit	fit	fit	fit	score		0.582
cut 5	fit	fit	fit	fit	fit	score	0.580

average the score of the one-feature model is **0.740**

The scored block always comes after the fitted rows. Each cut is a **fold**, and averaging over folds is **cross-validation**.

TimeSeriesSplit

Python Code

```
from sklearn.model_selection import TimeSeriesSplit

folds = TimeSeriesSplit(n_splits=5)

for fit_rows, score_rows in folds.split(train):
    print(len(fit_rows), "fitted", len(score_rows), "scored")
```

```
329 fitted 325 scored
654 fitted 325 scored
979 fitted 325 scored
1304 fitted 325 scored
1629 fitted 325 scored
```

cross_val_score

Python Code

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(LinearRegression(), train[A], train["vol_next"],
                        cv=folds, scoring="neg_root_mean_squared_error")

scores

array([-0.58322421, -0.70254508, -1.25463391, -0.58182607, -0.57991447])
```

The call fits five models and returns five scores. Each fold is scored by a model that was not fitted on it.

Why they are negative

scikit-learn reports every score so that **larger is better**. Errors are flipped to negative to fit that convention.

Python Code

```
print(-scores)
print(-scores.mean())
```

```
[0.58322421  0.70254508  1.25463391  0.58182607  0.57991447]
0.7404287479222657
```

A value of **-0.583** means an RMSE of **0.583**. Remove the minus sign before reporting it.

The five fold scores



Four of the folds give similar errors and one is roughly twice as large. That fold is scored on the period containing **March 2020**.

Two models can share an average and differ completely: one steady across five folds, another poor in one. Report the spread as well.

Choosing with cross-validation

Python Code

```
for columns in [A, B, C]:  
    scores = cross_val_score(LinearRegression(), train[columns], train["vol_next"],  
                             cv=folds, scoring="neg_root_mean_squared_error")  
    print(len(columns), "features:", round(-scores.mean(), 4))
```

```
1 features: 0.7404  
2 features: 0.7724  
3 features: 0.747
```

The **one-feature model has the lowest average error**. That result is an average over five cut dates rather than a single measurement.

Your turn 3

Build a fourth candidate that keeps `vol_20d` and `ret_20d` but drops `vol_60d`, and cross-validate it against the current best.

Python Code

```
D = [...]  
  
for columns in [A, D]:  
    scores = cross_val_score(LinearRegression(), train[columns], train["vol_next"],  
                             cv=folds, scoring="neg_root_mean_squared_error")  
    print(columns, round(-scores.mean(), 4))
```

Your turn 3: answer

```
D = ["vol_20d", "ret_20d"]
```

The errors are **0.7404** and **0.7477**. `vol_20d` on its own is still better, so `ret_20d` was not the column holding the larger models back.

None of the four beats the simplest. Say so in the report: the extra columns were tested and did not help.

6 • KFold and time order

PART 6

The standard form of cross-validation, and why it does not suit this table.

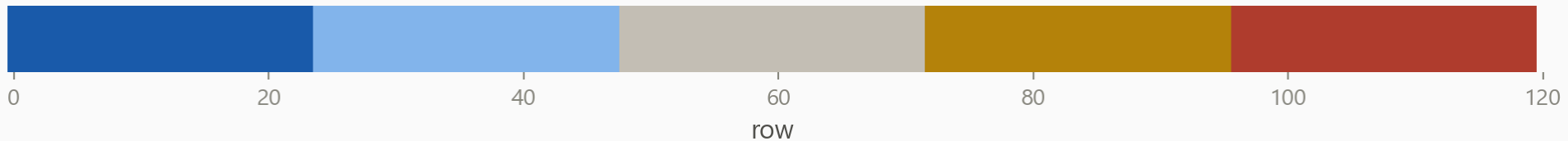
When the rows are not in time order

KFold shuffles the rows and cuts the folds anywhere, which assumes the rows are interchangeable. That holds for a table of loan applications, or of firms.

KFold: the folds are shuffled through the rows



TimeSeriesSplit: the folds move forward in time



KFold on the three candidates

Python Code

```
from sklearn.model_selection import KFold

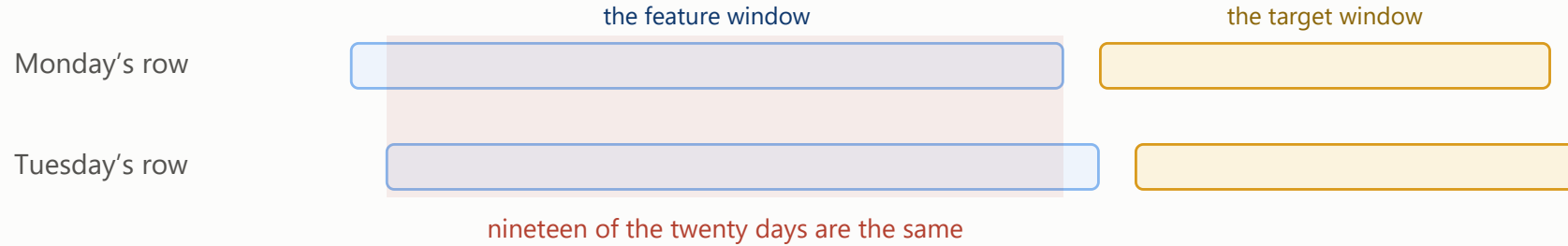
shuffled = KFold(n_splits=5, shuffle=True, random_state=0)

for columns in [A, B, C]:
    scores = cross_val_score(LinearRegression(), train[columns], train["vol_next"],
                             cv=shuffled, scoring="neg_root_mean_squared_error")
    print(len(columns), "features:", round(-scores.mean(), 4))
```

```
1 features: 0.7199
2 features: 0.7198
3 features: 0.7087
```

KFold gives the lowest error to the three-feature model, which had the **highest** error on the test rows.

Why shuffled folds flatter the score



Shuffling can put Monday in the fitting rows and Tuesday in the scored rows. They share nineteen of their twenty days, so Tuesday's score is far too good.

The same overlap, at the boundary

The last rows in a fitting block have targets that extend twenty days into the block used for scoring. The **gap** argument drops those rows.

Python Code

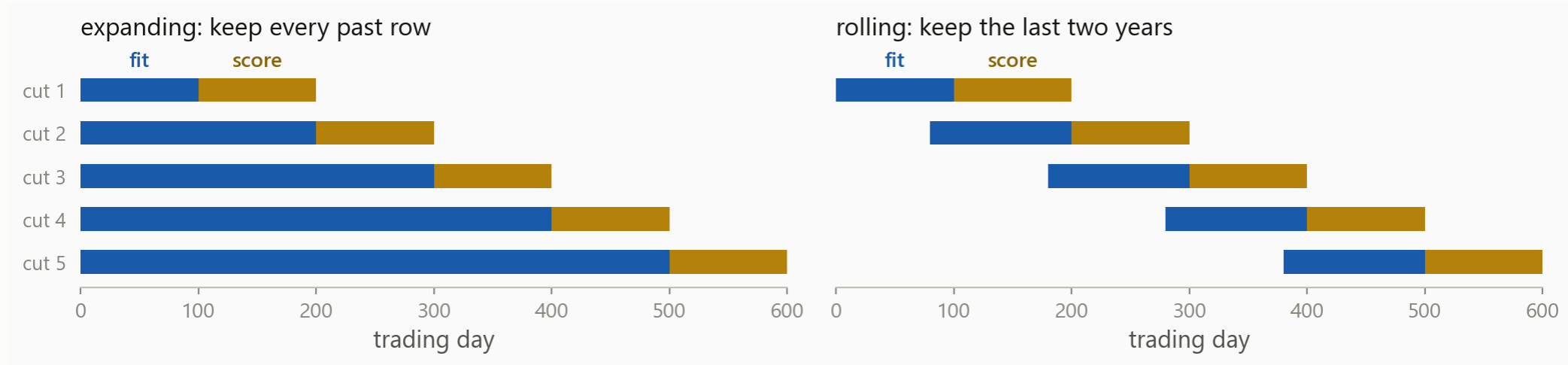
```
gapped = TimeSeriesSplit(n_splits=5, gap=20)

scores = cross_val_score(LinearRegression(), train[A], train["vol_next"],
                        cv=gapped, scoring="neg_root_mean_squared_error")
print(round(-scores.mean(), 4))
```

0.7394

The change is small because twenty rows are little against blocks of several hundred. It matters far more when the target spans a longer window.

Expanding and rolling windows



`max_train_size=500` produces the version on the right. Use a rolling window when older years have stopped being relevant, and an expanding window when they have not.

7 • More candidates than three

PART 7

Searching a long list of feature sets, and knowing when to stop.

Where the candidates come from

These three feature sets were written by hand. With p features there are 2^p subsets: a thousand at ten features, a million at twenty.

Fitting every subset stops being possible at about twenty features. The usual answer is to walk through the subsets one feature at a time instead.

Three ways to walk

Forward selection

start with nothing. Add the feature that lowers the error most, then the best of those left, and stop when it stops falling.

Stepwise selection

forward, but after each addition check whether an earlier feature has become redundant and can be dropped.

At ten features, forward selection fits 55 models instead of 1,024. None of the three is guaranteed to find the best subset, and all three usually get close.

Backward selection

start with every feature. Drop the one that costs least, and repeat. It needs more rows than features to get started.

AIC and BIC

Each walk still needs a rule for when to stop. AIC and BIC score a model by its fit **and** its size, so the two can be traded off in one number:

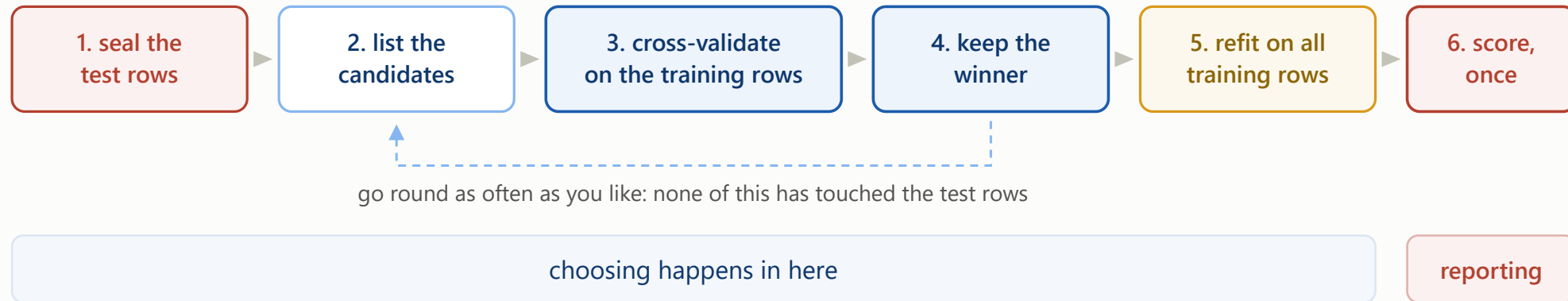
$$\text{AIC} = n \log(\text{MSE}) + 2d$$

$$\text{BIC} = n \log(\text{MSE}) + d \log(n)$$

- the first term falls as the fit improves, the second charges for each of the d parameters
- smaller is better, and adding a useless feature raises the total
- $\log(n)$ is larger than 2 here, so BIC charges more and prefers smaller models

One fit per candidate and no folds at all, which is why they are still used. Cross-validation is slower but assumes nothing about the shape of the errors.

The order it all happens in



Refit, then open the test rows

Cross-validation chose the one-feature model. Fit it on **all** the training rows, then score it once on the test rows.

Python Code

```
final = LinearRegression()
final.fit(train[A], train["vol_next"])

final_pred = final.predict(test[A])
print(np.sqrt(mean_squared_error(test["vol_next"], final_pred)))
```

```
0.40561048089369556
```

The folds were only a way of estimating this number. With the choice made, the model can use every training row.

The model against the two baselines

0.520

guess the average

0.417

repeat this month

0.406

the model selected by cross-validation

The model has the lowest error of the three, and the number can be defended: nothing in the selection used 2023 or 2024.

What stays with you

Fitting a model

`LinearRegression()`, `.fit(X, y)`, `.predict(X)`. Later models change only the first line.

Always report a baseline

Compare the model against the training average and against a simple rule that uses no model.

Choosing uses up data

Rows used to compare candidates can no longer give an honest score for the winner.

Read the folds, not only the mean

A fold much worse than the others points to a period the model handles badly.

X is a table, y is a column

`train[["vol_20d"]]` with two brackets, `train["vol_next"]` with one.

Training error always prefers more columns

Adding a column can never worsen the fit, so training error cannot say whether the column helps.

One split is one measurement

Moving the cut date can change the winner. Cross-validation averages over several cuts.

Folds must follow the calendar

Use `TimeSeriesSplit` for a time series. `KFold` shuffles, and here it chose the worst of the three models.

Where this goes next

before	features and targets, metrics, over- and underfitting	nothing fitted
now	fitting a model, and choosing between models honestly	scikit-learn
next	ridge and lasso, which add a setting you have to choose	the same folds, searched over a grid
then	classification, trees, ensembles	the same workflow

Every method later in the course has settings that have to be chosen. The procedure for choosing them is the one from this session.

Before the next session **CASE**

- **Exercises:** fitting, predicting, scoring against a baseline, and cross-validating a handful of candidates.
- **Case Part 5:** the risk report gets its first model, and has to justify the choice.

Stuck? Ask a friend, ask an AI for a hint rather than an answer, or email jobo@econ.au.dk.